APPFORGE / PRODUCT SPECIFICATION

AppForge Complete Requirements

One AI Studio handoff for your existing app ZIP and every accepted feature direction

SOURCE REVIEWED

Pasted text.txt - 2,633 lines / 107,557 characters

STATUS NOTE

Direct user requests are separated from follow-up directions, proposals, examples, and open decisions. Referenced app ZIP contents were not verified from this text attachment.

Prepared 13 November 2026

Contents

- 1. Purpose and handoff
- 2. Product goal and audience
- 3. Confirmed scope and interaction model
- 4. User workflow to support
- 5. Delivery and quality expectations
- 6. Recommended sequencing (assistant proposal, not a user-mandated order)
- 7. Open decisions to resolve from the repository
- 8. Requirements traceability checklist
- 9. Caveats about the pasted conversation
- 10. Copy-paste master prompt for Google AI Studio

1. Purpose and handoff

This brief consolidates the supplied 2,633-line pasted conversation and the user's follow-up context about giving an existing AppForge project ZIP and this PDF to Google AI Studio. It is intended to be uploaded alongside the actual project ZIP. It is not proof that the ZIP was inspected or that any suggested code in the conversation works.

What to give Google AI Studio

- Upload the complete AppForge source ZIP and this PDF in the same AI Studio project/chat.
- Paste the master implementation prompt in Section 10. If AI Studio cannot accept a PDF, paste the prompt and include the PDF's text content.
- Do not paste secret values into chat. Configure secrets only in the deployment platform's environment-variable settings. The screenshot in the conversation showed names such as `VERCEL_CLIENT_ID`, `VERCEL_CLIENT_SECRET`, and `APP_ENCRYPTION_SECRET`; it did not establish their values or that the user has a Vercel team ID. A team/account identifier is distinct from OAuth client credentials.
- Ask the coding assistant to inspect and map the uploaded ZIP before editing, then implement and report in stages. Do not ask it to replace the app with a fresh mockup.

Status labels

- **User-confirmed direction**: directly requested or explicitly accepted by a “yes”/“add all” reply in the conversation.
- **Assistant proposal accepted in principle**: a feature category followed by user acceptance; exact implementation detail still remains a proposal.
- **Example / implementation sketch**: illustrative code, libraries, paths, models, presets, and defaults appearing in assistant responses. These are not verified requirements for exact implementation.
- **Open choice**: the conversation does not settle a decision; inspect the project and propose options or ask before making a costly/irreversible choice.

2. Product goal and audience

AppForge is intended to help non-technical users create, visually edit, preview, export, and deploy frontend web applications. The experience should feel approachable to beginners while offering a more capable AI-assisted builder with live runtime and code workflows. The user wants these improvements added to their existing app and wants all distinct requirements retained when an AI coding assistant works from the project ZIP and this brief.

A prior assistant described the supplied app ZIP as binary/compressed and gave an architecture-oriented assessment, but the current source attachment is a pasted conversation, not the ZIP itself. No claims in this brief establish the actual state, framework, security, or completeness of the user's app.

3. Confirmed scope and interaction model

The user explicitly said they wanted to focus on all of the initial improvement areas, asked for capabilities comparable in ease and power to Lovable/Replit, said “add all this thing” for the five capability groups, and later said “yes” to additional groups. Treat the feature directions below as accepted scope. Do not silently downgrade them to optional ideas. The implementation details and sample code remain subject to repo inspection and the open choices in Section 9.

3.1 Project data and editing history

- Establish or preserve a structured project/page/node model so generated content remains editable as native AppForge elements.
- Support nested nodes with identifiers, type, properties, styles, and children; the examples use `ForgePage` / `ForgeNode` and node types `section`, `container`, `text`, `button`, and `3d-widget`.
- Add dependable undo/redo history for edits and AI changes; preserve project state and support keyboard shortcuts.
- Keep selection and updates tied to stable element IDs, including when AI changes a selected element.

Named implementation examples from the conversation (not binding choices)

The conversation's master prompt named React 19, TypeScript, Vite, Express (`server.ts`), Three.js, JSZip, Sandpack, Gemini (including `gemini-2.5-flash`), a 30-step `past/present/future` history stack, and keyboard shortcuts `Ctrl+Z`, `Ctrl+Y`, and `Ctrl+Shift+Z`. It gave preview widths of desktop 100%, tablet 768px, and mobile 375px; example wizard routes `/`, `/features`, `/contact`; sample vibe/theme presets; and possible files such as `src/utils/project-engine.ts`, `src/components/CanvasArea.tsx`, `src/components/PreviewMode.tsx`, and `src/utils/multi-framework-compiler.ts`. These are conversation-level implementation sketches. Inspect the uploaded repo and choose compatible values; preserve the intended functionality if the actual project uses different names or versions.

The deployment examples named Vercel OAuth callback `/api/auth/vercel/callback`, deployment proxy `/api/deploy/vercel`, GitHub sync `/api/sync/github`, AES-256-GCM token encryption, and secure HTTP-only cookies. These endpoint paths and cryptographic/session choices are not certified code; validate them against the existing backend and threat model.

3.2 Beginner visual editor

- Make it easy to start from a blank canvas, a guided example, a website portal, a mobile app, or prepared starting layouts.
- Support drag-and-drop or add/arrange/style workflows, click-to-select, clear selected-element feedback, and section reordering with an insertion indicator.
- Provide an inspector for selected elements: text, semantic heading tag, button label/link, colors, typography, padding, corner radius, and alignment.
- Include inline text editing and an AI “magic wand” to restyle or update the selected element while preserving its identity and leaving unrelated content alone.
- Include reusable smart sections such as hero, feature grid, pricing, contact, product showcase, and 3D showcase. Examples are not a closed component catalog.
- Provide global one-click theme presets and ensure generated themes preserve accessible text/background contrast.
- Provide desktop/tablet/mobile preview sizes, with responsive layouts that can be adjusted and checked.
- Keep the editor understandable for beginners and avoid requiring users to write code for ordinary visual tasks.

3.3 Prompt-to-app and image-to-app creation

- Add a guided app/site creation flow that asks for the project name, short description, audience, visual style/vibe, and needed pages/sections, then generates a coherent multi-page starting project and matching copy/theme.
- Make generated output editable in the visual canvas rather than a single flattened image or opaque blob.
- Add screenshot/sketch import: user uploads a UI screenshot or rough wireframe; vision AI infers layout, colors, and text and creates native editable blocks. Show errors and allow the user to inspect/modify the result.
- The prior assistant's fixed four-question wizard, three-page routes, theme labels, and API model are examples; use the existing app's architecture and confirm uncertain defaults.

3.4 AI copilot

- Keep an AI copilot docked beside the canvas so users can describe changes without leaving the builder.
- When a node is selected, scope the AI change to that node unless the user explicitly asks for a broader project change. With no selection, allow the assistant to generate/append a section or help at project level.
- Support streaming/progress and visible errors where feasible; generated changes should be reviewable and undoable.
- Require structured, schema-valid results that map to the project's native editable model. Validate generated JSON before applying it.
- The conversation named Gemini / Google AI Studio and an example Gemini model/API pattern. Exact model, API, and credential strategy are open choices and must be checked against the existing project and current platform support.

3.5 Live runtime and code sync

- Add a live runtime mode that can run the generated app with dependencies and surface runtime/build errors or console output.
- Add a code mode in which users can inspect/edit generated code and synchronize supported edits back into the visual node model.
- Provide an explicit mode switch among visual editing, live runtime, and code sync.
- Preserve a shared project representation/compiler path so preview, ZIP export, and deployment produce consistent results.
- The conversation proposed Sandpack/WebContainers and parsing HTML/JSX with `DOMParser`; these are implementation candidates, not verified technical decisions. Define supported syntax, parse failures, and safe fallback behavior before promising fully bidirectional conversion.

3.6 Data and working business flows

- Add a visual way to define data collections and generate typed CRUD integration/hooks for a selected backend provider.
- Offer practical contact/order/lead capture blocks, including email/contact forms and WhatsApp click-to-chat links, to reduce setup for beginners.
- Keep a clear distinction between a UI template, a working external integration, and a production-ready backend. Forms must report where submissions go and must not pretend to send data when no service is configured.
- Firebase/Firestore is the specific example used in the master prompt; Firebase vs Supabase vs other providers is unresolved. Web3Forms/Formspree were examples for hosted form delivery, not approved account setup or service commitments.

3.7 3D and assets

- Render interactive 3D widgets in editor preview and generated/exported apps, with responsive sizing and cleanup of WebGL resources.
- The examples name `floating-icosahedron`, `product-turntable`, `particle-wave`, and `glass-torus` presets and mouse/pointer interaction. These are sample presets, not a limit.
- Add a built-in image/icon/3D asset picker to reduce leaving the app to find assets.
- Asset records need usable source URLs, attribution/license metadata, and a clear policy for user-provided assets. The pasted example's Unsplash URLs were generic placeholders, not actual image assets or proof of licensing.

3.8 Compile, SEO, export, deploy, and sync

- Compile the project into a runnable frontend bundle with pages/routing, package manifest, build configuration, and static entry point.
- Provide one-click ZIP source export with a README explaining how to install and run locally.
- Add SEO basics: page metadata/Open Graph information, `sitemap.xml`, and `robots.txt` where suitable.
- Before publishing, run a preflight check for empty pages, placeholder links, and other actionable issues; show the findings and deployment status.
- Add a Vercel deployment flow with OAuth and server-side API proxying; show progress/results and useful errors.
- Add GitHub connection and two-way source sync/commits, plus visual version history/rollback. Clarify conflict resolution and commit behavior before implementation.

- The source alternates between one React/Vite bundle and a “multi-framework” compiler. A genuinely multi-framework compiler is not defined. Inspect the app, choose a truthful first supported target, and do not label a React-only compiler as multi-framework.

3.9 Security and reliability

- Never expose provider API keys, OAuth client secrets, access tokens, or encryption keys in browser code, generated source, public logs, or the downloadable ZIP.
- Route privileged OAuth/deploy/sync operations through a backend; store tokens with appropriate encryption and secure HTTP-only cookies where session architecture supports it.
- If user code or arbitrary widgets run in preview, isolate them using a restrictive sandbox and validate message origins/payloads. Do not grant iframe permissions without a specific need.
- Enforce project ownership and access control in the chosen backend; do not trust frontend checks as authorization.
- Validate AI output against schemas, sanitize/validate URLs and markup, and handle malformed output safely.
- Add recoverable errors, save-state feedback, and preflight/deployment failure messages.
- The conversation suggested AES-256-GCM, `allow-scripts` without `allow-same-origin`, and HTTP-only cookies. These are design sketches; have the implementation verify details against the app's actual threat model and runtime.

4. User workflow to support

1. Open AppForge and start a blank, guided, template, or image-import project.
2. Describe the site/app, audience, visual style, and desired pages; review the generated editable project.
3. Add/reorder smart sections, select elements, edit properties, apply a global theme, and use AI for scoped changes.
4. Preview at desktop/tablet/mobile sizes and optionally run the live runtime.
5. Inspect/synchronize code where supported; define backend data/forms only with explicit provider/configuration.
6. Run preflight; fix issues or consciously review them.
7. Export a runnable ZIP, deploy to Vercel, or sync with GitHub; review status and retain rollback/history options.

5. Delivery and quality expectations

- Work in the uploaded existing repository. First inventory its framework, scripts, data model, current features, auth/secrets handling, and deployment setup.
- Preserve working features, existing branding/design, and unrelated files. Avoid a wholesale rewrite unless inspection proves it necessary and the AI explains why.
- Map every checklist item in Section 8 to existing support, an implementation task, a blocked dependency, or an explicit open choice.
- Implement in small, coherent stages. After each stage, summarize changed files, behavior, validation performed, and remaining gaps.
- Do not claim that a provider integration, deployment, security property, test, or visual behavior works unless it was actually verified at the corresponding level.
- Do not leave placeholder stubs while claiming completion. If credentials or external accounts are unavailable, implement safe configuration surfaces and clearly identify the user-side setup needed.
- Preserve the full requirement set. If something cannot be implemented, list it with the reason instead of omitting or simplifying it.

6. Recommended sequencing (assistant proposal, not a user-mandated order)

1. Inspect repo and secure the foundation: schema, persistence, ID-stable edits, validation, undo/redo, secret handling.
2. Improve visual editor: canvas interactions, inspector, smart sections, responsive preview, themes.
3. Add scoped AI creation/editing and screenshot/sketch import with schema validation.

4. Add live runtime, error visibility, code sync, and 3D widgets.

5. Add data/forms/assets and clarify service configuration/licensing.

6. Unify compiler output; add SEO, ZIP export, preflight, Vercel deploy, GitHub sync, and version rollback.

Sequence can change if the inspected repository or dependencies justify it; keep the full accepted scope visible.

7. Open decisions to resolve from the repository

- What does the existing project already implement, and what framework, node schema, persistence, and compiler does it use?
- Is the first output target React/Vite only or are additional frameworks genuinely required?
- Which AI model/provider and secure key strategy fit Google AI Studio and the current app?
- Which database provider(s) will be supported first (Firebase, Supabase, or other), and what ownership/auth rules are required?
- What runtime powers the live preview (Sandpack, WebContainers, or existing infrastructure), and what are its dependency, resource, and isolation limits?
- What subset of JSX/HTML/CSS can round-trip safely between code and visual nodes?
- What Vercel OAuth app, callback URL, account/team scope, deployment project settings, and user consent flow are needed? Team ID must not be confused with OAuth Client ID/Secret.
- What GitHub sync model is desired: import/export, one-way commits, two-way sync, conflict resolution, and rollback semantics?
- Which external form/image/icon providers are selected, and what are their licensing, attribution, cost, quota, privacy, and credential requirements?
- Which required pages/templates, SEO defaults, accessibility target, supported browsers, and mobile behavior should define acceptance?
- Should authentication, payments, custom domains, billing, collaboration, and account management be included? These were raised as possible future add-ons, not accepted requirements in this source.

8. Requirements traceability checklist

Checklist status reflects conversation intent, not implementation status. All items remain **to inspect/implement/verify** in the uploaded codebase.

ID	Requirement to preserve	Status / source
R01	Inspect the existing ZIP and map architecture before edits; preserve working app	Confirmed by user's follow-up workflow; exact codebase unverified
R02	Structured editable pages/nodes with stable IDs, props, styles, children	Accepted technical direction; Forge schema is example
R03	Undo/redo history and keyboard shortcuts	Explicitly accepted initial improvement group
R04	Beginner-friendly project creation and visual editing	Explicitly accepted initial group and follow-up
R05	Drag/drop, click selection, inline text, section reorder	Accepted editor details; iframe code is sketch
R06	Properties inspector for text, links, style, spacing, alignment	Accepted PropertiesPanel proposal
R07	Scoped AI copilot and selected-element magic wand	Accepted Lovable/Replit capability and inspector proposal
R08	Schema-valid AI generation for editable project nodes	Accepted initial technical direction
R09	Guided prompt-to-site wizard with coherent pages/copy/theme	User said yes to added creation features
R10	Screenshot/sketch-to-editable-app import	User said yes to easy-life feature set
R11	Smart section library and global accessible themes	User said yes to feature set
R12	Responsive desktop/tablet/mobile preview	Accepted integrated workspace direction
R13	Live in-browser app runtime and console/error feedback	User said “add all this” to Lovable/Replit features

ID	Requirement to preserve	Status / source
R14	Visual/code sync with safe supported parsing	User said “add all this”; complete round-trip scope unresolved
R15	Visual data collections and generated CRUD integration	User said “add all this”; provider unresolved
R16	Working contact/order/lead capture including WhatsApp/email	User said yes to easy-life feature set; delivery services unresolved
R17	Interactive Three.js-style 3D widgets in editor and export	User said yes to 3D/widget direction; exact rendering requirements are examples
R18	Curated image/icon/3D asset picker with licensing metadata	User said yes to easy-life feature set; source/terms unresolved
R19	Consistent project compiler, ZIP export, runnable README	User said yes to ZIP/export direction
R20	SEO metadata, sitemap, robots files where applicable	User said yes to additional feature set
R21	Preflight validation and deployment status	Accepted initial technical direction
R22	Secure Vercel OAuth/deploy through server-side proxy	Accepted initial direction; credentials/account configuration unresolved
R23	GitHub OAuth, source sync/commits, visual history/rollback	User said “add all this”; exact sync/conflict model unresolved
R24	Secrets kept server-side; sandbox generated code; backend authorization	Explicitly accepted security improvements
R25	Implement in existing app without silently dropping scope; report gaps honestly	Direct user request in current task and follow-up
R26	User can give project ZIP plus this brief to AI Studio	Direct user follow-up; include in handoff instructions

9. Caveats about the pasted conversation

The pasted conversation contains illustrative component code, guessed file paths, example defaults, and API snippets. Some snippets visibly contain malformed or placeholder endpoint/URL constructions (for example Google API and WhatsApp URL strings), and the curated asset entries use generic Unsplash URLs. They must not be copied as production-ready code. The original discussion also presents Firebase, Sandpack/WebContainers, Vercel, GitHub, Web3Forms/Formspree, and specific React/Vite/Three.js/JSZip choices; those exact libraries/services should be confirmed against the uploaded source and current platform constraints.

The assistant's suggested additions of Stripe payment links, custom domains, and user-authentication templates were phrased as possible future fine-tuning. The user did not accept those in the supplied source; do not add them as requirements without asking.

10. Copy-paste master prompt for Google AI Studio

```text
You are a senior software engineer working inside my existing AppForge project. I have uploaded (1) the complete source-code ZIP of my current app and (2) the AppForge Consolidated Product Requirements PDF. Read the PDF and inspect the ZIP before editing any code. This is an upgrade to my existing application, not permission to discard it and generate an unrelated replacement.

PRIMARY GOAL Upgrade my existing AppForge into a beginner-friendly AI-assisted visual app/frontend builder that lets users create editable multi-page projects, edit them visually and with AI, preview them responsively and in a live runtime, inspect/synchronize code where supported, and export or deploy their work. Preserve every accepted requirement in the PDF. Do not omit, simplify, or silently defer requirements. If a requirement is blocked or ambiguous, keep it in the traceability checklist and explain the reason and decision needed.

FIRST: REPOSITORY DISCOVERY

1. Inventory the uploaded source tree and identify framework, package manager, scripts, app entry points, project/page/node model, persistence, current editor features, tests, deployment setup, backend, OAuth, and secret handling.

2. Summarize what already exists and map every PDF checklist ID R01-R26 to existing, missing, partial, blocked, or open-choice status. Cite relevant file paths.
```

3. Identify risks and conflicts with the current architecture before proposing changes. Do not expose secret values in output. Never ask me to paste API keys, OAuth secrets, tokens, or encryption keys into chat.

4. Give a concise staged implementation plan that preserves existing behavior. Then proceed with implementation in coherent stages, keeping the full requirements list visible. Do not stop after producing only a plan.

IMPLEMENTATION SCOPE

- Preserve/establish a validated editable page/node model with stable IDs, properties, styles, nested children, and reliable undo/redo.
- Make the editor beginner-friendly: project-start wizard; templates/smart sections; click selection; drag/drop/reorder; inline text edit; selected-element property inspector; accessible global themes; desktop/tablet/mobile previews.
- Add a docked AI copilot. Scope changes to a selected node when selected, validate structured output against the actual project schema, preserve IDs where appropriate, make edits undoable, and show errors. Add prompt-to-site generation and screenshot/sketch-to-editable-block import.
- Add a live runtime with useful errors/logs, code inspection/editing, and visual/code synchronization for a clearly documented supported syntax subset. Do not claim arbitrary JSX can round-trip if it cannot.
- Add visual data/CRUD generation with the provider choice documented; practical contact/order/lead forms including WhatsApp/email flows; and a built-in asset picker with real usable assets and license/attribution metadata. Never pretend an unconfigured form is sending data.
- Render responsive interactive 3D widgets in preview and compiled output; clean up resources and respect performance/accessibility settings.
- Use one consistent compilation pipeline for preview, runnable ZIP export, SEO outputs, and deployment. Include useful page metadata, sitemap/robots files where appropriate, preflight validation, deployment progress, Vercel deployment, GitHub sync/commit, and version history/rollback. Make conflicts and rollback behavior explicit.
- Keep API keys, OAuth credentials/tokens, and encryption secrets server-side. Secure backend routes, cookies, ownership checks, iframe/runtime isolation, message validation, generated markup/URLs, and AI output. Do not place secrets in frontend bundles or exported ZIPs.

DECISION AND SAFETY RULES

- Treat feature directions explicitly accepted by my “yes” / “add all” replies as required scope. Treat sample code, file paths, model names, presets, and provider choices in the PDF as examples unless the PDF explicitly marks them confirmed.
- Inspect the current repository before choosing libraries or replacing architecture. Prefer existing working dependencies where suitable. Call out React-only versus truly multi-framework support accurately.
- Resolve open choices from evidence where possible. If an account, paid service, external credential, destructive migration, or product decision is required, leave it safely configurable and tell me exactly what is needed. Never invent credentials or assume a team ID.
- Preserve the existing design and working features. Keep changes focused and do not remove unrelated files.
- Do not claim implementation, tests, deployment, provider connectivity, security, or visual behavior is verified unless you actually performed the relevant verification. If tools or credentials are unavailable, say so.

DELIVERY Implement the work in stages. After each stage, report changed files, user-visible behavior, checks actually run and their results, and any remaining blocker. Keep a requirements traceability checklist for R01-R26 with status and evidence. At completion, summarize what's implemented, what's partial or blocked, setup steps I must do, and how I can open/run the project. Do not silently drop any requirement. ```